Neurolabs Android SDK Integration Guide

Technical Integration Specification

Document ID: NLB-ANDROID-INTEGRATION-GUIDE
Version: v1.7.22
Date: 2026-10-18
Status: Partner integration reference
Owner: Neurolabs

1. Scope

This guide is for partner Android apps integrating ‘neurolabs-android-sdk‘ directly.

2. Release History

No public API contract has been broken since v1.1.9. New options have been added with backwards-compatible defaults; integrations on v1.1.9 keep working as is. Recommended defaults: strict shelf guidance with ‘validationPreset = IOS_PARITY‘, ‘liveQualityChecksEnabled = true‘, ‘autoCloseAfterCapture = true‘ for single-capture parity flows.

v1.7.22

No ‘NLConfiguration‘ changes and no removals. New APIs are additive, and the 1.7.21 ‘NLCustomCameraView‘ / ‘NLCaptureScreen‘ signatures stay as hidden overloads, so compiled hosts still link. One source change: ‘NLCaptureDeletionSource‘ gains ‘RETAKE‘, so a ‘when‘ over it with no ‘else‘ branch must handle the new entry.

If your app keeps photos from ‘onCapture‘:

- A retake now reaches ‘onCaptureDeleted‘ with source ‘RETAKE‘: drop that photo. A crop reaches the new ‘onCaptureCropReplaced‘ (a trailing parameter on ‘NLCustomCameraView‘ and ‘NLCaptureScreen‘, or ‘NeurolabsSDKCore.onCaptureCropReplaced‘ for ‘NLCaptureActivity‘) with the cropped photo: swap your copy of ‘replacedCaptureId‘. Both are experimental.
- A photo your app keeps and never enqueues is not recognised (that is iOS only). The suspend ‘NeurolabsSDKCore.recognizeCapture(captureId, imageData, detections)‘ recognises it without queueing it (section 11).

Recognition matches: the SDK recognises a capture where it is enqueued, which on Android mostly comes after ‘onCaptureComplete‘. Read them with ‘sdk.recognitionMatches(forCaptureId)‘ once the capture is enqueued; ‘CapturedImageData.recognitionMatches‘ on the delivered captures is filled only for a capture recognised by then. The multi-bay realogram’s ‘product_id‘ (‘NLCaptureActivity‘ rebuilds it after its save loop) and the visit KPIs now carry the matches.

Behaviour changes, with no opt-in:

- "Face the shelf straight on" (the shelf-row yaw leg) fires above 12 and clears at 9 or less (was 8 and 5). Every org’s ‘config_hash‘ in ‘guidance_config_applied‘ changes with this release.
- With ‘uploadsEnabled = false‘, a credential no longer fetches ‘GET /v1/config‘ for the queue or posts an owed mission submit.

- A commit recognises every detection whatever ‘sendDetectionsMetadata‘ says.
- A negative ‘NLDetectionOptions.maxDetections‘ keeps no detections, as on iOS, instead of throwing ‘IllegalArgumentException‘.
- The AR live "~ N products" chip counts SKU facings only, and shutter presses during limited ARCore tracking are not fused. Served counts are unchanged.

New APIs: ‘NLCaptureCropReplacement‘, ‘onCaptureCropReplaced‘ (camera parameter and core), ‘NeurolabsSDKCore.recognizeCapture(...)‘, and ‘NLARSessionRecording‘ (experimental, armed process-wide with ‘NLARSessionRecording.armed‘).

v1.7.21

No ‘NLConfiguration‘ changes. New APIs are additive. One breaking removal: ‘NLOnnxYoloDetector‘, ‘NLOnnxYoloConfig‘ and ‘ModelExportInstructions‘ are gone. The ONNX detector was a stub that never returned a detection; if your app called it directly, use the SDK’s built-in detection instead.

Behaviour changes, with no opt-in:

- An embedded camera (‘NLCustomCameraView‘, ‘NLCaptureScreen‘) that leaves the composition (a navigation push, a tab switch) pauses and resumes on return; its photos are kept. The session ends on close, Done, a new ‘sessionId‘ or the Activity finishing (section 7).
- A camera that cannot start reports ‘onError‘ with the new code ‘CAMERA_UNAVAILABLE‘, once per session. It is not sent to Sentry.
- ‘CapturedImageData.validationResult‘ and ‘NLNativeCaptureResult.validationResult‘ carry the quality verdict (null when nothing validated the capture).
- Image quality reads luma as Rec.601, as on iOS: a saturated colour can get a different verdict than on 1.7.20. No threshold changed. With both overlays off and no preset the camera uses ‘LENIENT‘, as iOS does, and ‘minDetectionCount‘ (‘FRIDGE‘, or a custom value) gates captures.
- The org’s quality knobs, capture-image size and guidance disables follow the remote config during a session; a host’s own values still win.
- Photos of an SDK capture session killed before Done are held and offered at the next SDK capture screen ("Unsaved photos", Save or Discard), not uploaded on their own.

New APIs (section 12):

- Store context: ‘prefetchStoreContext(outletIds)‘, ‘storeContextStatus(outletId)‘, ‘setStoreContextIdentity(identity)‘, ‘clearStoreContextCache()‘.
- Visit KPIs: ‘NeurolabsSDKDelegate.onVisitKpis(result, sessionId)‘ (default no-op), ‘setVisitKpisEnabled(enabled)‘ (on by default), ‘setVisitKpiContext(sessionId, context)‘, ‘computeVisitKpis(sessionId, captures, multiBay, context)‘.

v1.3.2

- Mixpanel ingestion endpoint now resolves via ‘NLAnalyticsDefaults.MIXPANEL_ENDPOINT_EU‘ at SDK construction; events flow into the EU region project as expected. The standalone ‘MixpanelAnalyticsTracker‘ class keeps the US endpoint as its default so partners reusing it for their own US-region projects are unaffected.

- ‘CaptureViewModel.onFinishPressed‘ now claims ‘isFinishing‘ synchronously, wraps every disk read (pending review reload, capture reload, temp-file cleanup) in ‘withContext(Dispatchers.IO)‘, and offloads ‘spillCaptureToDisk‘ from ‘commitCapture‘ - the Done button stays responsive throughout multi-capture sequences and shows an inline ‘CircularProgressIndicator‘ while persistence is in flight.
- Embedded custom-camera demo gained an in-app settings sheet (gear icon, top-right of the camera). Sliders for detection thresholds, capture min / max, live-quality FPS; switches for every boolean knob; SharedPreferences-backed persistence across launches.

v1.3.1

- Fix for an uncatchable native ‘SIGSEGV‘ in ‘com.google.ai.edge.litert.Model.nativeLoadAsset‘ that triggered when the TFLite asset was stored compressed in the APK, missing, or zero length. ‘NLLiteRTYoloDetector‘ now probes the asset with ‘AssetManager.openFd‘ first and, on failure, extracts the asset to ‘noBackupFilesDir‘ and loads it via the file-path overload.

v1.3.0

- Relaxed guidance thresholds (perspective skew floor raised, angle-off warning relaxed) for less aggressive blocking during normal in-store motion. Android lint passes restored.

v1.2.8

- Onboarding flow: same-barcode relookup is allowed after a completed attempt so partners can retake without restarting the session.

v1.2.7

- Lint compatibility fix for product capture; no API change.

v1.2.6

- Onboarding: partner-overridable operations lookup client and capture debug exports (frame dump for offline review).
- Demo: scrollable launcher on small devices, nav-bar inset respected on the product-capture back button.

v1.2.5

- ‘NLNativeCaptureConfig‘ / ‘NLCustomCameraView‘ expose ‘showsSequenceCounterLabel‘ to toggle the on-screen sequence counter.

v1.2.4

- New capture range controls: ‘allowManualFinish‘, ‘minCapturesBeforeDone‘, and ‘maxCaptures‘ plumbed through ‘NLNativeCaptureConfig‘ for bounded multi-capture flows with a Done button.
- iOS-camera parity work: tap-to-focus, lens-switcher hardening, tightened quality validation thresholds.

v1.2.3

- ‘NLNativeCaptureConfig.cameraMode = AR | DEFAULT‘ lets partners pick between the ARKit/ARCore pipeline and the AVFoundation/CameraX 0.5/1 lens-switcher path.

- ‘init‘ accepts ‘analyticsEnabled‘, ‘sentryEnabled‘, ‘sentryDsn‘ so partner apps can opt out of telemetry or point Sentry at their own project.
- Default validation thresholds (blur, glare, perspective) tightened.

v1.2.2

- Internal: Android plugin integration handling refactor; no public API change.

v1.2.1

- iOS camera-style UI parity polish, lens-switcher state machine hardening, demo Sentry auto-init disabled.

v1.2.0

- Internal scaffolding for v1.2.x: capture range controls landed in v1.2.4, lens switcher in v1.2.4-v1.2.5, missions policy preset, retry + idempotency conformance fixture.

v1.1.9 (baseline)

- No public API contract break.
- Default task routing config uses ‘taskUUID‘.
- Recommended custom-camera shelf payload remains strict shelf guidance with ‘validationPreset = IOS_PARITY‘.
- ‘liveQualityChecksEnabled‘ remains the key switch for pill / rotation / warning / error guidance behavior.
- ‘autoCloseAfterCapture = true‘ remains the recommended parity flow for single accepted captures.

3. Requirements

- ‘minSdk 26‘
- JDK 17
- Kotlin 2.x

4. Install

```
// app/build.gradle.kts
dependencies {
    implementation(files("libs/neurolabs-android-sdk-v1.1.x.aar"))
}
```

If you consume a local ‘.aar‘, include required runtime deps (Compose/CameraX/serialization/etc.) or use the provided integration gradle script from the Cordova plugin setup.

5. SDK Initialization + Warmup

```
import ai.neurolabs.sdk.core.*
import kotlinx.coroutines.launch

class MainApp : Application() {
    override fun onCreate() {
        super.onCreate()

        val config = NLConfiguration.production(
            apiKey = BuildConfig.NL_API_KEY,
            taskUUID = BuildConfig.NL_TASK_UUID,
            debugLogging = BuildConfig.DEBUG
        )

        NeurolabsSDKCore.configure(this, config)
    }
}

// e.g. in first Activity/VM
val sdk = NeurolabsSDKCore.shared ?: return
lifecycleScope.launch {
    sdk.initialize()   // the SDK's one detector, all_in_one
    sdk.waitForFullyLoaded()
}
```

The detector is downloaded, not bundled (1.7.13+). The SDK ships no model, so it adds about 6.5 MB to your app. Its one detector, ‘all_in_one‘ (YOLO11l-Seg at 960 px, 12 classes: SKUs, ‘sinc‘ multipacks, prices, promos, posters, gaps, every row type and ‘shelf_strip‘), comes from the Operations API, per org, with the operations key (or auth provider) you already configure: a signed URL, a sha256-verified, resumable, cached 55.6 MB download. The SDK prefetches it at start-up, when the credential changes and when a capture session opens. The old model names (‘NLBModel.tflite‘, ‘yolo11l_seg_sam.tflite‘, ...) and every ‘NLYoloModelVariant‘ now mean ‘all_in_one‘.

The first launch needs network for detection. Until the model is on the device, capture works without a detector: the shutter, the queue and the light/glare/contrast checks work, ‘waitForFullyLoaded()‘ returns, ‘modelPreparationState‘ stays ‘LOADING‘, detection throws the retryable ‘DETECTOR_UNAVAILABLE‘, and the capture screen shows "Detector downloading. You can still capture." (no boxes, no "Point at a shelf", no AR anchors). When the model lands it is swapped in mid-session. ‘NLDetectorModels.status(context)‘ gives the state and, after a failure, the failure and its code in ‘reason‘, the same codes as iOS: ‘no_credentials‘, ‘not_entitled‘, ‘unknown_model‘, ‘pin_mismatch‘, ‘sha_mismatch‘ (an hour before the API is asked again) and ‘api_unavailable‘, ‘download_failed‘ (60 s; ‘compile_failed‘ is iOS-only), plus ‘insufficient_storage‘ (no wait: the next capture session or ‘prefetch‘ asks again) and ‘load_failed‘ (a fresh copy that still will not load; not retried until the next launch). New operations credentials clear the wait. With no operations credential and no model on the device, the SDK does not fetch on its own and the status stays ‘NOT_DOWNLOADED‘ until a credential is set (1.7.14). To have it before the first session (on Wi-Fi, after the operations credential is set):

```
lifecycleScope.launch { NLDetectorModels.prefetch(context) }
```

Keep the SDK's own download off cellular (1.7.16+). By default the SDK downloads the detector on any network. Set ‘NLDetectorModels.prefetchPolicy = NLDetectorPrefetchNetworkPolicy.UNMETERED_ONLY‘ before ‘NeurolabsSDKCore.configure(...)‘ and its own downloads (start-up, credential change, a waiting ‘initialize‘, a capture session opening) wait for an unmetered network (Wi-Fi, Ethernet; a carrier's temporarily unmetered network only with Data Saver off) and start by themselves when one becomes the default network. Meanwhile ‘status‘ is ‘NOT_DOWNLOADED‘ with ‘reason‘ ‘waiting_for_unmetered_network‘, and the camera runs without a detector ("Detector unavailable. You can still capture."). An explicit ‘NLDetectorModels.prefetch(context)‘ ignores the policy, so call it when you do want the cellular download (for example when the rep opens the camera with no detector yet).

Allow ‘api.operations.neurolabs.ai‘ and ‘storage.googleapis.com‘ in any egress allowlist. The detector reads its input size and class order from the model, and live frames are sized to it. It costs about 2.25 the pixels per frame of a 640-px model, so check the live frame rate on your lowest-end device first.

6. Queue Management

```
val sdk = NeurolabsSDKCore.shared ?: return

// runtime queue behavior
sdk.setAutoSyncEnabled(true)
sdk.setWifiOnlyUploadsEnabled(false)
sdk.setDeletePhotosOnUploadSuccess(true)
sdk.setMaxQueueSize(200)
sdk.setUploadRetryCount(5)

// lifecycle actions
lifecycleScope.launch {
    sdk.flushQueue()
    sdk.retryFailedUploads()
}

val status = sdk.getQueueStatus()
println("pending=${status.pendingCount} failed=${status.failedCount}")
```

‘retryFailedUploads()‘ gives failed photos a fresh cycle, except a mission refusal that cannot succeed as sent (‘wrong_status‘, ‘submission_mismatch‘, ‘mission_not_found‘, ‘invalid_subsection_name‘, ‘invalid_subsection_id‘, or any ‘not_claimed_by_*‘); ‘flushQueue(force = true)‘ and ‘retryQueuedItems(ids)‘ still revive those. Since 1.7.20 a call within 60 s of the previous one does nothing, so a retry button or timer cannot reset every backoff; the SDK reports the calls hourly (‘queue_manual_retry‘).

Inspecting the queue at launch

After ‘configure()‘ the queue loads the records an earlier process left on disk in the background. Until that pass ends, ‘getQueuedItems()‘ can be empty or partial, and ‘removeQueuedItem(id)‘ or ‘clearUploadQueue()‘ can miss records that are still on disk. Before you inspect, withdraw or discard queued work at launch, wait for the pass:

```
import ai.neurolabs.sdk.core.awaitQueueHydrated

lifecycleScope.launch {
    sdk.awaitQueueHydrated() // the whole pass; returns at once once it has run
    sdk.getQueuedItems()
        .filter { it.sessionId == abandonedSessionId }
        .forEach { sdk.removeQueuedItem(it.id) }
}
```

- It returns once the whole pass has finished, also when the load failed (the queue then holds what it could read).
- A setter that rebuilds the queue (‘setOperationsAuthApiKey‘, ‘setOperationsBaseUrl‘, ‘setMaxQueueSize‘, ‘setUploadRetryCount‘) starts a new pass; the call waits for the queue that is current when it returns.
- On an instance that was released (‘release()‘, or replaced by a ‘configure()‘ with a different configuration) it throws ‘IllegalStateException‘. Call it on the instance ‘configure()‘ returned.

Visits, owners and submits (1.7.15)

A host that tracks visits, or signs several users in on one device, can use these calls (see the 1.7.15 changelog entry for the full rules):

```
import ai.neurolabs.sdk.core.*

lifecycleScope.launch {
    // One queue per user and environment instead of per apiKey. Returns the new core;
    // set credentials on it. The previous owner’s work stays on the device, untouched.
    val sdk = NeurolabsSDKCore.switchQueueOwner(context, config, "prod:$userId")

    // Asked once when a call is refused (409/403) because the mission’s claim was lost.
    sdk.setOperationsReclaimHandler { missionId -> reclaimMission(missionId) }
    launch { sdk.submitEvents.collect { event -> /* NLSubmitEvent.Submitted / Abandoned */ } }
}

    sdk.openSession(sessionId)        // the orphan sweep never submits it on its own
    sdk.openOperationsSessions()      // at launch: stop or discard each open session
    sdk.queuedSessions()              // every session until its submit is confirmed
    sdk.owedSubmits()                 // submits the SDK still owes
    sdk.retryQueuedItems(listOf(id))  // retry chosen failed items only
    sdk.discardOperationsSession(sessionId) // or discardAllOperationsWork()
}
```

- Each queued photo carries ‘NLQueuedItem.missionId‘ (the session’s mission, stamped at enqueue), ‘lastErrorCode‘ / ‘lastErrorMessage‘ and ‘bytesSent‘.
- ‘clearUploadQueue()‘ drops queued items and notes and, since 1.7.20 (as on iOS), the uploaded-but-unsubmitted image ids with the submits still owed for them. It does not wait for uploads in flight; ‘discardAllOperationsWork‘ does (‘discardOperationsSession‘ drops one visit).
- The embedded camera (‘NLCustomCameraView‘, ‘NLCaptureScreen‘) takes ‘onCapture: ((CapturedImageData) -> Unit)?‘, called once per photo as it joins the session; key copies by ‘captureId‘.

Kept photos and low storage (1.7.16)

Uploaded photos no longer take a ‘maxQueueSize‘ slot, in any mode. A host that keeps photos after upload (‘setDeletePhotosOnUploadSuccess(false)‘) or for a submit can bound them (see the 1.7.16 changelog entry for the full rules):

```
import ai.neurolabs.sdk.core.NLConfiguration

NLConfiguration.maxRetainedPhotos = 500        // default; a warning only, never refuses a ca
pture
NLConfiguration.retainedPhotoExpiryDays = 14   // default null (off); photos kept for a submi
t never expire
```

- From 80% of ‘maxRetainedPhotos‘ the SDK calls ‘NeurolabsSDKDelegate.onRetainedPhotosHigh(retained, limit)‘ once per crossing. ‘NLQueueStatus.retainedCount‘ / ‘retainedBytes‘ report the kept photos.
- A new capture is refused with ‘NLCaptureErrorCodes.STORAGE_LOW‘ (retryable) when the queue’s storage has under 500 MB free. It reaches the host through ‘onError‘ and the failed ‘onCaptureResult‘, as ‘QUEUE_FULL‘ does.
- Since 1.7.20 the SDK’s own camera asks the queue before it takes a photo: a photo that would be refused for ‘STORAGE_LOW‘ or ‘QUEUE_FULL‘ is not taken, the camera tells the rep why, and the host gets ‘onError‘ with no capture id (no failed ‘onCaptureResult‘, since there is no photo).

Uploads after the app is backgrounded

The queue’s WorkManager drain keeps uploading while the app’s process lives, also in the background. After the process is killed, uploads resume when the host next initialises the SDK (for most apps, the next open). iOS differs: a flush there gets iOS’s background grace period (about 30 s), then resumes on the next foreground.

What reaches the backend

What the operations upload carries:

- Per capture: the JPEG (the full frame, sized by ‘maxImageDimension‘ / ‘imageCompressionQuality‘), with its task and bay (‘subsection_name‘).
- Per capture, after the JPEG uploads: its capture metadata (capture contract v1, ‘PUT .../images/{imageId}/capture-metadata‘), for captures the SDK camera took: shutter time, SDK/device/OS, camera mode, photo size, an EXIF subset (no GPS, no serial numbers), device attitude, camera intrinsics and, on the AR camera, its pose. The detector’s boxes only with ‘sendDetectionsMetadata‘. Location only with ‘NLConfiguration.captureGeolocation‘ AND a location permission your app declares (‘ACCESS_FINE_LOCATION‘ or ‘ACCESS_COARSE_LOCATION‘) and holds; the SDK declares none and never asks. The ARCore depth map only with ‘NLConfiguration.captureDepth‘. Both default off. It goes up from its own retrying outbox and never holds up the photo or the submit. Your app reads the same record on ‘NLNativeCaptureResult.captureMetadata‘.
- Per session: the mission submit’s ‘client_metadata.device_dedup‘ summary for a multi-bay session (section 8).

What stays on the device:

- The capture's detections always reach your app ('CapturedImageData.detections', 'onCapture', the capture result); 'sendDetectionsMetadata' decides only whether they are kept in the queue record and uploaded with the capture metadata.
- 'enablePreviewCropping' trims only the review thumbnail, to the area the detections cover. The uploaded photo is always the full frame, as on iOS. Up to 1.7.17 Android uploaded the trimmed photo.
- AR session recordings ('.nlrec', the AR component's recording API) are tooling for replay and calibration. They are not a capture output and are never uploaded.

7. Open Custom Camera (Native Capture UI)

```
import ai.neurolabs.sdk.models.*
import ai.neurolabs.sdk.ui.openNativeCaptureUI

val sessionId = UUID.randomUUID().toString()

val config = NLNativeCaptureConfig(
    guidanceMode = NLCaptureGuidanceMode.STRICT,
    validationPreset = NLValidationPreset.IOS_PARITY,
    confidenceThreshold = 0.25f,
    iouThreshold = 0.45f,
    maxCaptures = 5,
    allowManualFinish = true,
    minCapturesBeforeDone = 2,
    showsCapturedCountLabel = false,
    showAlignmentGuidance = true,
    enableValidation = true,
    autoCloseAfterCapture = true,
    showDetections = false,
    showCapturedRegions = false,
    liveQualityChecksEnabled = true,
    liveQualityTargetFps = 6
)

val options = NLNativeCaptureOptions(
    type = NLShelfType.SHELF,
    taskUUID = "<TASK_UUID_FOR_THIS_SESSION>",
    sendDetectionsMetadata = true,
    enablePreviewCropping = true,
    maxImageDimension = 1920,
    imageCompressionQuality = 0.85f
)

sdk.openNativeCaptureUI(
    context = this,
    sessionId = sessionId,
    config = config,
    options = options
)
```

Recommended capture payload notes:

- 'liveQualityChecksEnabled = true' is the key for pill/rotation/warning/error guidance behavior.
- Keep 'type = NLShelfType.SHELF' and 'guidanceMode = NLCaptureGuidanceMode.STRICT' for full shelf guidance checks.
- Keep 'showDetections = false' and 'showCapturedRegions = false' to stay on the custom-camera guidance UI path.

- ‘guidanceMode = NLCaptureGuidanceMode.GUIDANCE‘ should only be used as a temporary fallback if a partner specifically needs looser Android behavior while a strict-guidance regression is being fixed.

End-of-session review (1.7.16)

‘finalReview = true‘ (on ‘NLNativeCaptureConfig‘ or ‘NLNativeCaptureOptions‘, options first; or ‘NLCameraConfiguration.finalReview‘) gives the default (non-AR) camera without a multi-bay plan one review at the end instead of a per-shot preview:

- Done, or the count where the session would end today (‘maxCaptures‘, the auto-close target, or the required count), opens "Review". A tap on a photo opens it with Crop, Delete and Previous / Next. Delete fires ‘onCaptureDeleted‘ with source ‘REVIEW‘.
- Nothing is queued or uploaded before Confirm. The photos are staged on disk, so they survive the process being killed. Confirm queues the remaining photos in capture order, crops applied.
- Default ‘false‘: every other session, and every host that does not set it, is unchanged.

Embedded camera: who owns the photo files (1.7.19)

‘NLCaptureActivity‘ / ‘openNativeCaptureUI‘ queue the photos themselves; nothing below applies to them. When you embed the camera (‘NLCustomCameraView‘, ‘NLCaptureScreen‘), ‘onCaptureComplete‘ hands the session’s photos to you:

- Each ‘CapturedImageData.imageFilePath‘ stays on disk after the camera leaves the composition. Read it, show it, or pass its bytes to ‘enqueueCapture‘ (which copies them into the upload queue) whenever you are ready, including after you have closed the camera.
- The files are yours from then on. Release them once you are done: ‘sdk.acknowledgeCaptures(sessionId)‘ for the whole session, or ‘sdk.releaseCapture(captureId)‘ one by one.
- Files you never release are deleted at the next SDK start or capture session once they are 24 hours old, so disk cannot leak.
- Photos the session discarded before ‘onCaptureComplete‘ (a retake, a deletion, before Done) are never delivered; the SDK deletes them.

```
NLCustomCameraView(
    sdk = sdk,
    sessionId = sessionId,
    onClose = { showCamera = false },
    onCaptureComplete = { captures ->
        showCamera = false                        // closing first is safe
        scope.launch(Dispatchers.IO) {
            captures.forEach { capture ->
                sdk.enqueueCapture(sessionId, capture.captureId, capture.resolveImageData(),
capture.detections, NLCaptureContext())
            }
            sdk.acknowledgeCaptures(sessionId)   // done with the files
        }
    },
)
```

Before 1.7.19 the camera deleted these files when it left the composition, so a host that enqueued or read them after closing it lost the photos without an error.

Embedded camera lifecycle (1.7.21)

- Leaving the composition (a navigation push, a tab switch) pauses the embedded camera: the camera closes and the session's photos stay. Coming back reopens the camera and the live checks. The session ends on close, Done, a new ‘sessionId‘, or the Activity finishing.
- A configuration change (dark mode, font scale, multi-window, a fold) rebinds the camera to the recreated Activity, so the preview stays live.
- A camera that cannot start raises ‘onError‘ with ‘NLCaptureErrorCodes.CAMERA_UNAVAILABLE‘ once per session and tells the rep "The camera could not start. Another app may be using it."
- ‘CapturedImageData.validationResult‘ (‘isAcceptable‘, ‘overallScore‘, ‘issues‘) carries the quality verdict; null when nothing validated the capture. A strict failure never reaches ‘onCapture‘.
- The SDK unbinds only its own CameraX use cases, so a camera your app runs (a barcode scanner, say) keeps running.

Offline chip (1.7.17)

The default and AR camera show a small chip while the upload queue sees no network: "Offline uploads queued" (or "Offline" with nothing queued), and "Waiting for Wi-Fi uploads queued" while Wi-Fi-only uploads hold photos on a metered network. It hides while the network is unknown and when the host uploads the photos itself (‘uploadsEnabled = false‘). It is informational only and never gates the shutter. On by default; turn it off with ‘NLCameraConfiguration(showsOfflineIndicator = false)‘, or on the bridge with ‘showsOfflineIndicator‘ (‘Boolean?‘) on ‘NLNativeCaptureOptions‘ / ‘NLNativeCaptureConfig‘ (options, then config, then ‘true‘).

AR boxes the live labels have not matched (1.7.17)

Under ‘arBoxStyle = NLARBoxStyle.HIGHLIGHT‘, with ‘arLiveLabels‘ on and a ready recognition provider, a stable box that has been looked at and has no product name latched on it (no match, or every look below 0.6) is drawn in ‘arBoxUnmatchedColor‘ (‘#RRGGBB‘, default ‘#FACC15‘) instead of ‘arBoxHighlightColor‘. A box not looked at yet keeps the highlight colour. The key is on ‘NLCameraConfiguration‘ and, as ‘String?‘, on ‘NLNativeCaptureConfig‘ / ‘NLNativeCaptureOptions‘ (options, then config, then the default). Nothing changes while ‘arLiveLabels‘ is off.

With recognition off there are no product names or unmatched colour, and a box is told apart from its neighbours by its position, not by which product it is: two identical facings that touch can be harder to keep apart. From 1.7.18 appearance helps there; see the next subsection.

AR box identity with recognition off (1.7.18)

With on-device recognition off, the AR overlay tells touching boxes apart by appearance as well as position. ‘CaptureScreenConfig.arAppearanceWithoutRecognition‘ (default ‘true‘; ‘Boolean?‘ of the same name on ‘NLNativeCaptureConfig‘ / ‘NLNativeCaptureOptions‘, options, then config, then ‘true‘) uses the same ConvNeXt model as recognition, downloaded for appearance only through ‘GET /v1/device-models‘ for an org Operations grants it to (on-device recognition on, or the org’s ‘ar_appearance_embedder‘ flag), and a colour histogram until the model is on the device, if it cannot be fetched, or for any other org. ‘NLDetectorModels.prefetchPolicy‘ applies to the download. Only the AR camera starts it; the default camera never does.

Full-resolution AR stills (1.7.18, experimental)

‘arHighResolutionStills = true‘ (on ‘CaptureScreenConfig‘ and ‘NLCameraConfiguration‘; ‘Boolean?‘ on the bridge types, then ‘false‘) takes each AR photo as a JPEG still of up to 4032 px through ARCore’s Shared Camera, beside tracking, instead of ARCore’s CPU image (about 2.1 MP). The org’s ‘sdkCapture.arHighResolutionStills = false‘ vetoes it. It needs API 29+ and a Shared Camera session the phone accepts; anything else, and any failure at capture time, keeps the CPU image for that photo, so a capture never fails because of it. Two failed stills in a row switch it off for the session. Off by default until the device spike passes.

Names for the live labels come from ‘NLCameraConfiguration.productDetailsSource‘ (an ‘NLProductDetailsResolver‘). It is in-process only, never in the bridge’s JSON: it reaches ‘NLCustomCameraView(cameraConfiguration = ...)‘ and ‘NLCaptureActivity.open‘ / ‘createIntent(context, sessionId, cameraConfiguration)‘. (iOS sets the same source on ‘NeurolabsSDKCore.productDetailsSource‘ for ‘openNativeCaptureUI‘.)

8. Multi-Bay Capture (Long Shelves)

Multi-bay capture handles shelves too long for a single photo: the user side-steps along the shelf and captures one overlapping photo per bay, the SDK guides the side-step, dedups the overlap on device, and reports merged product counts across bays in a single session result.

Enable it by setting ‘multiBay‘ on the capture config (‘null‘ = feature off, the default):

```
import ai.neurolabs.sdk.models.*
import ai.neurolabs.sdk.multibay.NLMultiBayOptions
import ai.neurolabs.sdk.ui.openNativeCaptureUI

val config = NLNativeCaptureConfig(
    guidanceMode = NLCaptureGuidanceMode.STRICT,
    multiBay = NLMultiBayOptions(
        minBays = 2,                    // minimum bays before the session can finish (default
 1)
        maxBays = 4,                    // maximum bays per session (default 8; cap 8, or 30 f
or a continuous AR sweep)
        targetOverlapFraction = 0.30,  // fraction of the previous bay kept visible (default
0.30)
        minDetectionConfidence = 0.5,  // confidence floor for merge/count participation (def
ault 0.5)
        onDeviceDedup = true,           // run overlap dedup at session finish (default true)
        generatePreview = true,         // compose the display-only stitched preview (default
true)
        previewMaxDimension = 2048     // preview long-edge cap (default 2048, clamped 512..4
096)
    )
)

sdk.openNativeCaptureUI(
    context = this,
    sessionId = sessionId,
    config = config
)
```

The same ‘multiBay‘ field exists on ‘NLCameraConfiguration‘ for the custom-camera config path, and on the WebView/Cordova bridge payload (camelCase field names identical across iOS, Android, and Cordova; absent fields take the defaults above).

The bay cap depends on the session’s mode: a continuous sweep (‘captureMode = NLCaptureMode.CONTINUOUS‘) on the AR camera may plan up to ‘NLMultiBayOptions.MAX_SUPPORTED_CONTINUOUS_BAYS‘ (30) bays; every other session, and a continuous sweep that falls back to CameraX, keeps 8. A value outside the cap is clamped and logged when the camera opens. From v1.7.17 the WebView bridge resolves ‘maxBays‘ the same way; in v1.7.16 it clamped to 8 on every mode. Cordova’s ‘NLCaptureActivity‘ launch already followed the mode in v1.7.16.

"Review shelf" for long sweeps and "On this shelf" (1.7.17)

- Past 8 bays the AR flow’s review shows the shelf as pages of consecutive bays (9 5 + 4, 30 8 + 8 + 7 + 7), each stitched when the rep turns to it, with Previous / Next and a swipe. Up to 8 bays nothing changes, and ‘NLMultiBayResult.previewImageData‘ is still the one panorama.
- The same review lists the walk’s facings by brand, with each brand’s share of the recognised products, and by SKU. A facing counts as recognised at a match of 0.6 or more. It needs on-device recognition (section 11) and a ‘productDetailsSource‘ for names and brands; without recognition it shows the product total only.
- No API change.

Reading merged results:

- Activity Result / bridge: ‘NLNativeCaptureResultPayload.multiBay‘ (‘NLNativeMultiBayResultSummary?‘, null unless the session ran multi-bay) carries ‘countsByLabel: Map<String, Int>‘, ‘baysCount‘, ‘dedupTrusted‘, and ‘previewImageFileUri‘. It also carries the optional ‘realogram‘ (‘NLStitchedRealogram‘ with camelCase keys: ‘version‘, ‘coordinateSpace‘, ‘sdkVersion‘, ‘bays‘, ‘shelves[{shelfIndex, rows[{rowIndex, slots[{slotIndex, xMin, yMin, xMax, yMax, bayIndex, detectorClass, trackId, productId, stackIndex, score}]}]}]‘, ‘truncated‘, ‘omittedProductCount‘) and ‘segments‘ (‘[{fromBay, toBay, trusted, countsByLabel}]‘), each left out when the SDK built none. The realogram never crosses Binder: the WebView bridge carries it inline, while the Activity result carries ‘realogramFileUri‘ instead, a ‘file://‘ URI to ‘<cacheDir>/neurolabs/realogram-<sessionId>.json‘ holding the same JSON. Read it, then delete it; the SDK deletes any left older than 24 h at init.
- Bridge: ‘arAppearanceWithoutRecognition‘ (‘Boolean?‘) on ‘NLNativeCaptureOptions‘ / ‘NLNativeCaptureConfig‘ resolves options, then config, then ‘true‘, into ‘CaptureScreenConfig.arAppearanceWithoutRecognition‘. ‘arHighResolutionStills‘ (‘Boolean?‘) resolves the same way, then ‘false‘.
- Compose: ‘NLCaptureScreen‘ / ‘NLCustomCameraView‘ accept an ‘onMultiBayResult: ((NLMultiBayResult) -> Unit)?‘ callback delivering the full result - ‘bays: List<NLBaySummary>‘, ‘mergedProducts: List<NLMergedProduct>‘ (one entry per physical product instance after cross-bay dedup), ‘countsByLabel‘, the stitched preview (‘previewImageData‘), and ‘dedupTrusted‘.
- Use ‘NLMultiBayResult.productCount‘ for a product tally - it sums only ‘sku_single‘ + ‘sku_multipack‘; ‘countsByLabel‘ also carries price/promo/poster buckets, so summing every label over-counts products.
- ‘countsByLabel‘ (‘counts_by_label‘ on the wire) is keyed by detector class (‘sku_single‘, ‘sku_multipack‘, price tag, promo, poster), not by product or SKU. Shelf, row and slot positions are in ‘NLMultiBayResult.realogram‘ (‘NLStitchedRealogram‘).

Meaning of ‘dedupTrusted‘:

- ‘true‘ - counts and ‘mergedProducts‘ are on-device deduped: each physical product instance is counted once across overlapping bays.
- ‘false‘ - an adjacent-pair alignment broke, the dedup timed out, or ‘onDeviceDedup‘ was disabled; counts are then the per-bay sum (an upper bound), not a deduped count.

Segments and the stitched realogram (1.7.18)

- A broken link between two adjacent bays no longer leaves every later bay unmerged: it starts a new segment, and the dedup runs inside each run of linked bays. ‘NLMultiBayResult.segments‘ (‘NLDedupSegment‘: ‘fromBay‘, ‘toBay‘, ‘trusted‘, ‘countsByLabel‘) lists them. ‘dedupTrusted‘ is still true only for one trusted segment. When every segment is trusted and they tile the walk, the submit sends ‘client_metadata.device_dedup‘ v2 (‘version: 2‘, ‘segments‘); otherwise v1. "Review shelf" marks each broken link.
- ‘NLMultiBayResult.realogram‘ (‘NLStitchedRealogram‘) groups the walk’s merged products into shelves, rows and slots in the first bay’s frame, by geometry alone, so it exists with recognition off. SDK-managed flows send it as the mission submit’s top-level ‘realogram‘; a custom capture UI’s ‘submitNotesJSON()‘ never carries it. Older backends ignore it.

Registration, retakes and the shelf's bay order (1.7.19)

- Adjacent bays register with a full homography (rotation, scale and perspective), as on iOS, so a rep who turns, steps in or tilts between bays keeps the link. ‘ref_homographies‘ keep their shape. Registration has a 1.5 s share of the merge's 4 s timeout: on a slow phone a long walk breaks its last links instead of losing the whole merge, and "Review shelf" marks such a break "Bay N not linked - split here to keep up".
- A link that broke because the photos did not register is marked "Couldn't link bay N - retake it to join", with "Retake bay N". The camera opens with a "Retaking bay N" banner; the next photo replaces bay N in place (same bay index) and the review merges again. A retake is offered only while the session still holds the bay: a bay already handed to the upload queue cannot be replaced.
- A walk that folds back across a broken link (the next bay revisits an earlier one, within 0.75 m by AR position) is registered to that earlier bay, so its products count once. CameraX walks, which have no AR positions, make no attempt.
- A split walk's ‘device_dedup‘ v2 also sends ‘seams‘ (one per break, with the device's bound on products counted on both sides) and, with them, ‘links‘. Every multi-bay submit whose bays have their spatial inputs sends ‘bay_order‘, v1 or v2 (the bays left to right on the shelf; bay indexes everywhere else keep their capture order). The bays reach the merge and the submit in capture order (before 1.7.19, parallel commits could shuffle them).

Upload semantics:

- Each bay photo uploads as a normal capture through the operations queue (resumable mission flow, one image per capture).
- On ‘stopSession‘ the mission submit attaches a compact merge summary (‘{bays, counts_by_label, dedup_trusted, ref_homographies, sdk_version}‘) as structured ‘client_metadata.device_dedup‘ plus, transitionally, a ‘notes‘ duplicate for older backends. The backend stores the summary and serves ‘counts_by_label‘ only when ‘dedup_trusted‘ is true. It does not re-dedup from ‘ref_homographies‘ today.
- SDK-managed capture flows record that summary automatically. Only hosts presenting their own custom capture UI must pass ‘NLMultiBayResult.submitNotesJSON()‘ to ‘sdk.setMultiBaySubmitNotes(notes, sessionId)‘ before ‘stopSession‘.

Config-only mode:

- There is no partner-facing in-camera mode toggle. The Single Multi-bay capture-mode chip and sheet are internal demo/debug chrome gated behind ‘NLSDKDebug.internalCameraToolsEnabled‘ (default ‘false‘; never enable it in partner builds). Partners control the mode purely via configuration: ‘multiBay‘ set = multi-bay session, ‘multiBay = null‘ = single-bay session - programmatic ‘multiBay‘ configuration is unaffected by the gate.
- The Neurolabs demo app enables that chrome only in local debug builds; Firebase/release demo builds default to single-bay exactly like partner builds. The multi-bay feature itself is fully available in all builds via configuration - only the demo defaults changed.

9. Post-Processing Hooks

Use Activity Result API and process captures when camera returns.

```
private val captureLauncher = registerForActivityResult(
    NLCaptureActivity.Contract()
) { result ->
    when (result) {
        is NLCaptureActivityResult.Success -> {
            // capture metadata is in result.payload
            val count = result.payload.captures.size
            Log.d("NLB", "camera returned with $count captures")
        }
        is NLCaptureActivityResult.Cancelled -> {
            Log.d("NLB", "camera cancelled")
        }
        is NLCaptureActivityResult.Error -> {
            Log.e("NLB", "camera error: ${result.error.message}")
        }
    }
}
```

10. Delegate/Event Callbacks

```
sdk.delegate = object : NeurolabsSDKDelegate {
    override fun onCaptureResult(result: NLNativeCaptureResult, captureId: String, sessionId:
 String) {}
    override fun onError(error: NLError, sessionId: String?, messageId: String?) {}
    override fun onQueueStatusChange(status: NLQueueStatus) {}
    override fun onLoadingProgress(progress: NLLoadingProgress) {}
    override fun onQueueItemEnqueued(item: NLQueuedItem) {}
    override fun onUploadSucceeded(item: NLQueuedItem) {}
    override fun onUploadFailed(item: NLQueuedItem, error: NLError) {}
}
```

11. On-Device Recognition (v1.7.1)

Per-account, config-gated: your operations API key resolves YOUR org's recognition config and vector dataset - enabling an account is a backend flip, no app change.
Every stage degrades gracefully; capture is never blocked by recognition.

```
import ai.neurolabs.sdk.recognition.NLRecognitionBootstrap

val bootstrap = NLRecognitionBootstrap(
    context,
    NLRecognitionBootstrap.Configuration(
        operationsApiKey = "<your operations key>",
        operationsBaseUrl = "https://api.operations.neurolabs.ai/v1",
    ),
)
bootstrap.start(sdk, applicationScope) // safe to call again; see below
// bootstrap.status (StateFlow)  "provider: ready (N items)" etc.
// bootstrap.statusSummary  also says when the chain was built offline
```

Offline. Every build that completes online saves the recognition config and the snapshot's signed manifest for that operations key (the file is named by a one-way fingerprint of the key; the key is never stored). When a later build cannot reach the Operations API (no signal, a timeout, a captive portal, a 5xx), it builds from that record for up to the org's offline expiry after the last full sync (‘offline_ttl_days‘ in the recognition config, 1 to 30 days; else ‘bootstrap.offlineCacheTTL‘, 7 days; records saved before 1.7.19 read as 7 days), as long as the embedder and every snapshot shard are still on the device and pass their sha256 checks. ‘bootstrap.offlineSnapshotDate‘ is then the time of that sync, and ‘statusSummary‘ reads ‘provider: ready (offline, 42 items, snapshot from 2026-10-01)‘. A 401 or 403 still switches recognition off and forgets the record.

Short-lived tokens. If your bearer is a session token rather than a fixed key, build the configuration with your ‘NLOperationsAuthProvider‘ and a stable identity for the rep or org instead:

```
NLRecognitionBootstrap.Configuration(
    authProvider = myAuthProvider, // e.g. NLRefreshableAuthProvider { exchangeSessionToken()
 }
    offlineIdentity = repId,       // stable across tokens; must not be blank
    operationsBaseUrl = "https://api.operations.neurolabs.ai/v1",
)
```

Every request takes the provider's current token. A 401 drops it and the request is made once more with a fresh one; only a second 401 switches recognition off and forgets the record. The offline record is named after a one-way hash of ‘offlineIdentity‘ (never stored as is), so a record saved under yesterday's token serves today's, and a cold start with no network (and so no token) still builds from it. A provider that cannot give a token counts as offline. A blank identity throws ‘IllegalArgumentException‘. ‘copy()‘ on this configuration returns an API-key configuration: build a new one instead.

Retries. A build that ended unavailable is not final: calling ‘start‘ again builds again once 60 s have passed after a network-type failure, or 1 h after a definitive one (disabled, refused, mismatched). After a network-type failure the bootstrap also rebuilds by itself when the device's network comes back (at most 5 times per instance). A later ready build injects its provider into the SDK like the first.

The embedder ‘.tflite‘ (DINOv3 ConvNeXt-Tiny) downloads through the Operations API (‘GET /v1/device-models/dinov3-convnext-tiny-android-tflite-v1‘, sha256 and size pinned in the SDK) once recognition-config has enabled recognition for your org, and is cached, so later launches need no network. It is the only embedder the SDK runs: a model your app ships as an asset is never used. When the model cannot be had, recognition is off with the reason, for example ‘embedder unavailable (not_entitled)‘. The release AAR carries the Qdrant Edge native libraries; the default index remains the built-in brute-force this release.

Tap-to-product-card: captures expose per-detection matches via ‘sdk.recognitionMatches(forCaptureId:)‘; pair the SDK's ‘DetectionOverlay(onDetectionTap = ...)‘ with ‘NLDetectionInspectorSheet(detection, match, resolver, onDismiss)‘ and an ‘NLProductDetailsResolver‘ built over ‘NLOperationsCatalogClient‘. Offline or unresolved products fall back to a UUID + similarity row.

Matches on the captures (v1.7.22): the SDK recognises a capture where it is enqueued, mostly after ‘onCaptureComplete‘ (the ‘NLCaptureActivity‘ save loop, or your own ‘enqueueCapture‘ call), so read the matches with ‘sdk.recognitionMatches(forCaptureId:)‘ once the capture is enqueued. A delivered capture’s ‘CapturedImageData.recognitionMatches‘ is filled only when it was recognised before delivery. The multi-bay realogram (rebuilt after ‘NLCaptureActivity‘’s save loop) and the visit KPIs read the stored matches. To recognise a photo outside the camera (one you stored yourself), call the suspend ‘sdk.recognizeCapture(captureId, imageData, detections)‘. It returns the matches and keeps them for ‘recognitionMatches(forCaptureId:)‘; null means recognition did not run (no provider, the kill switch on, a provider not ready, or an image that does not decode), and an empty map means it ran and matched nothing. Nothing is queued or uploaded.

12. Notes

Store context for visit KPIs (1.7.21)

A visit’s KPIs need the outlet’s store context (targets, assortment, planograms and the org’s metric definitions). The SDK fetches, verifies and caches it per credential for the org’s offline TTL (default 7 days). Outlet ids are Operations outlet UUIDs. The calls are extensions on ‘NeurolabsSDKCore‘ (‘import ai.neurolabs.sdk.core.*‘).

```
val result = sdk.prefetchStoreContext(routeOutletIds)  // never throws; result.refreshFailure
 is a code
val status = sdk.storeContextStatus(outletId)           // cache only, no network
sdk.setStoreContextIdentity(currentUser.id)  // only with setOperationsAuthProvider; hashed,
never stored or sent
sdk.clearStoreContextCache()                  // sign-out
```

‘status.state‘ is ‘READY‘, ‘EXPIRED‘ or ‘MISSING‘, and ‘status.reason‘ is a stable code (‘not_assigned‘ means the outlet id is not your org’s; treat an unknown code as missing). ‘device_kpis‘ are submitted only for a context that is ready and ‘isSignatureVerified‘; the SDK pins the production key ‘store-context-v1‘ and requires signatures. The staging key ‘store-context-v2‘ is trusted only on a Neurolabs staging operations host. The full code list is in the README ("Store Context (visit KPIs)").

Visit KPIs (1.7.21)

After an SDK-managed session (‘NLCaptureActivity‘, the bridge, Cordova) finishes with at least one capture, the SDK computes the visit’s KPIs on the device and calls ‘NeurolabsSDKDelegate.onVisitKpis(result, sessionId)‘ (default no-op). On by default; ‘setVisitKpisEnabled(false)‘ turns it off.

```
sdk.setVisitKpiContext(sessionId, NLVisitKpiContext(outletId, sectionsByCaptureId = emptyMap(
), sectionsByBayIndex = emptyMap()))
// before the session finishes; refreshes the store context
val result = sdk.computeVisitKpis(sessionId, captures, multiBayResult, context)  // your own
capture UI; read-only
```

‘NLVisitKpiResult‘ carries the canonical ‘nl.visit_kpis‘ v1 JSON (‘payloadJson‘), ‘isProvisional‘ (always true on the device: label it "Provisional"), ‘storeContext‘ (‘Used(offline)‘, ‘Missing‘, ‘Expired‘), ‘perfectStoreScoreBp‘, ‘actionCount‘ and ‘willSubmit‘. Without a store context, show gaps and price-tag presence only, as "Needs store targets". The payload rides the mission submit as ‘client_metadata.device_kpis‘ only when the store context was ready and signature-verified and the payload passes the backend’s checks; a ‘400 invalid_device_kpis‘ makes the SDK resubmit once without it. Details are in the README ("Visit KPIs").

Telemetry and privacy (Sentry)

The SDK reports errors to the Neurolabs Sentry project under this policy, the same on Android, iOS and Cordova:

- **Only the SDK’s own errors.** The SDK reports the errors it captures itself. It never calls ‘SentryAndroid.init‘ or ‘Sentry.init‘: its client is private, so no uncaught-exception handler, ANR, network, session or lifecycle integration is installed in your app, a crash or ANR in your own code is never sent, and your own Sentry setup is untouched.
- **No credentials or personal data.** ‘isSendDefaultPii‘ is off. Messages are scrubbed before they are sent: a URL keeps only its scheme, host and path, so a signed URL’s credentials never leave the device. Since 1.7.20 an error that came from a server response carries only the HTTP status and the server’s ‘error_code‘, never the response body.
- **Opt out** with ‘sentryEnabled = false‘ (or ‘analyticsEnabled = false‘) on ‘NLConfiguration‘. Nothing is sent from a unit or instrumented test run.

Other notes

- Per-session upload routing is supported with ‘options.taskUUID‘.
- ‘autoCloseAfterCapture=true‘ closes after Save from preview/review flow.
- Use ‘allowManualFinish = true‘ with ‘minCapturesBeforeDone‘ and ‘maxCaptures‘ when you want "at least X, up to Y" capture sequences.
- Set ‘showsCapturedCountLabel = false‘ to hide the ‘N captured‘ chip in the top bar.
- For partner apps, avoid passing base64 images across process boundaries.